```
class Turtle
  forward: (distance) ->
    # moves the turtle distance in the direction it is facing
    this
  rotate: (degrees) ->
    # rotates the turtle 90 degrees clockwise
    this
  move: ({direction, distance}) ->
    # moves the turtle in a given direction
    this
  stop: ->
    # stops the turtle
```

The reason for making the methods evaluate to this is explained later in this section.

The `forward` method is invoked with an integer that specifies how far forward the turtle should move. The `rotate` method takes an integer that is the number of degrees that the turtle should rotate *clockwise*—by turning right. To make the turtle walk around a 10 x 10 square, Scruffy has to give it seven commands:

```
turtle = new Turtle
turtle.forward 10
turtle.rotate()
turtle.forward 10
turtle.rotate()
turtle.forward 10
turtle.rotate()
turtle.forward 10
```

When asked about this, Agtron suggests that Scruffy can extend the API by adding a `square` method to the prototype (see chapter 5):

```
Turtle::square = (size) ->
  @forward size
  @rotate()
  @forward size
  @rotate()
  @forward size
  @rotate()
  @forward size
```

Notice that when adding a square method, you use the @ symbol to invoke a method on the turtle object that the method was invoked on.

Or with brevity:

```
Turtle::square = (size) ->
  for side in [1..4]
```